

# Post-Quantum Broadcast Authentication and Verifiable Delay Functions for IEC 61850 Industrial Control Systems

Prabakaran Kannan

Research Scholar, Department of Computer Science and Engineering  
National Institute of Technology Puducherry  
Email: cs23d2005@nitpy.ac.in

**Abstract**—Industrial control systems (ICS) governing electrical substations, chemical plants, and critical manufacturing rely on the IEC 61850 protocol suite for real-time multicast communication. The Generic Object Oriented Substation Event (GOOSE) and Sampled Values (SV) protocols transmit protection and measurement data with sub-millisecond latency requirements, yet their native multicast model provides no source authentication, leaving them vulnerable to spoofing and replay attacks. Simultaneously, safety instrumented systems (SIS) depend on software-enforced timing delays—emergency shutdown cooldowns, interlock holds, and purge cycles—that an attacker with sufficient computational resources can bypass by fast-forwarding timer logic.

This paper presents two complementary constructions that address these threats in a post-quantum setting. First, we introduce TESLA++, an extension of the Timed Efficient Stream Loss-tolerant Authentication (TESLA) protocol that replaces classical primitives with quantum-resistant alternatives: SHAKE256-based one-way hash chains, HMAC-SHAKE256 message authentication codes, and ML-DSA-65 digital signatures for chain commitment bootstrapping. We specify three protocol profiles—GOOSE (10 ms intervals,  $<200\ \mu\text{s}$  authentication), SV (250  $\mu\text{s}$  intervals,  $<50\ \mu\text{s}$  MAC computation), and MMS (100 ms intervals)—each calibrated to IEC 61850 timing constraints. Second, we propose a Verifiable Delay Function (VDF) construction based on iterated SHA3-256 hashing that produces non-parallelizable, efficiently verifiable timing proofs for safety-critical operations. Five safety profiles are defined covering SIL 2 and SIL 3 requirements with delays ranging from 10 to 120 seconds, accompanied by ML-DSA-65 signed attestations for regulatory audit trails. Security analysis demonstrates that TESLA++ achieves source authentication under the one-wayness of SHAKE256 and unforgeability of ML-DSA-65, while the VDF construction guarantees sequentiality under the random oracle model. Performance evaluation on representative industrial hardware shows MAC computation latencies of 38  $\mu\text{s}$  for SV and 142  $\mu\text{s}$  for GOOSE, with VDF verification completing in under 0.8 ms regardless of the original computation duration.

**Index Terms**—Post-quantum cryptography, IEC 61850, TESLA, broadcast authentication, verifiable delay functions, SCADA security, functional safety, ICS

## I. INTRODUCTION

The digitalization of electrical substations, process plants, and manufacturing facilities has placed the IEC 61850 communication standard at the center of modern industrial automation. IEC 61850 defines three principal service mappings—GOOSE for event-driven protection signaling, SV for contin-

uous measurement streaming at rates up to 4 kHz, and MMS for supervisory control—that together form the communication backbone of substation automation systems (SAS) [?], [?]. These protocols operate over switched Ethernet with strict real-time constraints: a GOOSE trip command must propagate across a protection zone in under 4 ms, while an SV sample must arrive within a 250  $\mu\text{s}$  window [?].

The multicast nature of GOOSE and SV communication introduces a fundamental security challenge. Because messages are broadcast to all subscribers on a LAN segment, any device with network access can inject forged frames. Demonstrated attacks include GOOSE spoofing that triggers false circuit-breaker trips [?], SV injection that corrupts phasor measurements [?], and replay attacks that exploit the absence of per-message authentication. The 2015 attack on the Ukrainian power grid underscored the real-world consequences of inadequate ICS security [?].

IEC 62351-6 specifies security extensions for IEC 61850, including message authentication codes (MACs) for GOOSE and SV frames [?]. However, the current specification relies on HMAC-SHA-256 with symmetric key distribution, offering no mechanism for source authentication in multicast settings—any subscriber holding the group key can forge messages attributed to any sender. Furthermore, all cryptographic primitives specified in IEC 62351 are vulnerable to quantum computing attacks. Shor’s algorithm can break the RSA and ECDSA signatures used for key management, while Grover’s algorithm reduces the effective security of symmetric primitives [?], [?].

A second, orthogonal concern arises in safety instrumented systems governed by IEC 61508 and IEC 61511 [?], [?]. These systems enforce mandatory timing delays—emergency shutdown cooldowns, interlock holds, chemical reaction holds, and purge cycles—that prevent premature restart of hazardous equipment. When these delays are implemented purely in software, an attacker who compromises the safety controller can bypass the timer logic entirely, creating conditions for catastrophic physical harm. No existing ICS security standard provides a cryptographic mechanism to guarantee that a minimum physical time has elapsed.

## A. Contributions

This paper makes the following contributions:

- 1) **TESLA++ Protocol.** We extend the TESLA broadcast authentication protocol [?], [?] with post-quantum primitives, replacing SHA-1/HMAC-SHA-1 with SHAKE256/HMAC-SHAKE256 and adding ML-DSA-65 chain commitment signatures. We define three optimized profiles for GOOSE, SV, and MMS traffic with demonstrated compliance to IEC 61850 timing budgets.
- 2) **VDF-Based Safety Timing Proofs.** We propose a verifiable delay function construction based on iterated SHA3-256 that produces efficiently verifiable proofs of elapsed time. We define five safety profiles aligned with IEC 61508 SIL levels and present a hardware calibration methodology that maps iteration counts to physical delay durations.
- 3) **Security Analysis.** We provide formal security arguments for both constructions: source authentication for TESLA++ under standard hash function assumptions, and sequentiality for the VDF under the random oracle model.
- 4) **Performance Evaluation.** We present comprehensive benchmarks demonstrating that TESLA++ meets all IEC 61850 timing constraints and that VDF verification completes in sub-millisecond time regardless of the original delay duration.

The remainder of this paper is organized as follows. Section II provides background on IEC 61850, TESLA, VDFs, and relevant safety standards. Section III defines the system model and threat model. Section IV presents the TESLA++ construction. Section V describes the VDF-based timing proofs. Section VI provides security analysis. Section VII reports performance results. Section VIII surveys related work, and Section IX concludes.

## II. BACKGROUND

### A. IEC 61850 Communication Architecture

IEC 61850 organizes substation communication into three logical levels interconnected by a station bus and a process bus [?]. At the process level, merging units digitize current and voltage transformer outputs into SV streams transmitted at 4 kHz (4000 samples per second, one sample every 250  $\mu$ s) over multicast Ethernet [?]. At the bay level, intelligent electronic devices (IEDs) exchange protection signals via GOOSE multicast messages with a transfer-time budget of 3–4 ms for critical trip commands [?]. At the station level, MMS provides client-server communication for supervisory control with relaxed timing on the order of hundreds of milliseconds.

**GOOSE Protocol.** Generic Object Oriented Substation Events carry binary status changes (e.g., breaker position, protection relay trips) as ASN.1-encoded PDUs directly over Ethernet (EtherType 0x88B8). Upon a state change, the publisher retransmits the GOOSE frame with exponentially increasing retransmission intervals (e.g., 2, 4, 8, ... ms) to ensure

delivery despite frame loss. Each GOOSE message includes a state number (StNum) and sequence number (SqNum) but no cryptographic authentication.

**Sampled Values Protocol.** SV frames carry digitized analog measurements (currents, voltages, power quality data) as a continuous stream. At 4 kHz with 8 channels of 32-bit samples, the raw data rate reaches approximately 1 Mbit/s per stream. The tight timing requires that any authentication overhead remain below 50  $\mu$ s per sample to avoid violating the 250  $\mu$ s sample interval.

**MMS Protocol.** The Manufacturing Message Specification provides confirmed services over TCP/IP for reading/writing data objects, reporting, and control operations. Timing requirements are on the order of 100 ms to seconds, permitting more computationally expensive security mechanisms.

### B. TESLA Broadcast Authentication

The Timed Efficient Stream Loss-tolerant Authentication (TESLA) protocol, introduced by Perrig et al. [?], [?], provides asymmetric-like source authentication for multicast streams using only symmetric cryptographic primitives. The core idea exploits delayed key disclosure combined with one-way hash chains.

**Hash Chain Construction.** The sender generates a one-way hash chain  $\{K_0, K_1, \dots, K_n\}$  where  $K_n$  is a randomly chosen seed and  $K_i = H(K_{i+1})$  for  $i = n-1, \dots, 0$ . The anchor  $K_0$  is authenticated via a digital signature and distributed to all receivers.

**Authentication.** Time is divided into intervals of duration  $\Delta$ . During interval  $j$ , the sender authenticates each message  $m$  by appending  $\text{MAC}(K_j, j \| m)$ . The key  $K_j$  is not yet known to receivers at this point.

**Disclosure.** After a delay of  $d$  intervals, the sender discloses  $K_j$ . Receivers verify the disclosed key by checking  $H^j(K_j) = K_0$  (walking the hash chain down to the anchor) and then verify the buffered messages' MACs using  $K_j$ .

**Security.** The one-way property of  $H$  ensures that receivers cannot compute future keys from disclosed past keys. Since  $K_j$  is unknown to receivers at the time the message is sent, only the sender (who pre-computed the full chain) can produce a valid MAC, achieving source authentication.

### C. Verifiable Delay Functions

A Verifiable Delay Function (VDF) is a function  $f : \mathcal{X} \rightarrow \mathcal{Y}$  that requires a prescribed number of sequential steps to evaluate, yet whose output can be efficiently verified [?]. Formally, a VDF satisfies three properties:

*Definition 1 (VDF Properties):* A VDF with parameter  $T$  provides:

- 1) **Sequentiality:** Any algorithm computing  $f(x)$  requires at least  $T$  sequential steps, even with polynomial parallelism.
- 2) **Efficient verification:** Given  $x$ ,  $y = f(x)$ , and a proof  $\pi$ , verification runs in time  $O(\text{polylog}(T))$ .
- 3) **Uniqueness:** For every input  $x$ , there exists exactly one valid output  $y$ .

Wesolowski [?] and Pietrzak [?] proposed efficient VDF constructions based on repeated squaring in groups of unknown order. In our setting, we construct a simpler VDF from iterated hash function evaluation, which is inherently sequential due to the data dependency between consecutive hash invocations.

#### D. IEC 62351-6 Security

IEC 62351-6 [?] defines security mechanisms for IEC 61850 profiles. For GOOSE and SV, it specifies optional HMAC-based authentication appended to frames. However, the standard relies on pre-shared symmetric keys distributed via IEC 62351-9 Group Domain of Interpretation (GDOI), which provides group-level authentication but not source-level authentication within the group. All group members share the same MAC key, meaning any compromised IED can forge messages attributed to any other IED.

#### E. Safety Integrity Levels

IEC 61508 [?] defines four Safety Integrity Levels (SIL 1 through SIL 4) that specify target failure rates for safety functions. SIL 3 requires a probability of dangerous failure on demand between  $10^{-4}$  and  $10^{-3}$ . Safety instrumented systems (SIS) conforming to IEC 61511 [?] implement safety functions including emergency shutdowns, pressure relief, and interlock management, all of which incorporate mandatory timing delays to prevent hazardous premature restarts.

### III. SYSTEM MODEL AND THREAT MODEL

#### A. Substation Architecture

We consider a typical IEC 61850-based substation with the following components:

- **Merging Units (MUs):** Digitize analog CT/VT outputs and publish SV streams at 4 kHz over the process bus.
- **Protection IEDs:** Subscribe to SV streams and publish GOOSE trip commands. Each IED protects a bay (feeder, transformer, busbar).
- **Bay Controllers:** Execute control sequences and publish GOOSE status messages.
- **Station Gateway:** Provides MMS access to the SCADA master and engineering workstations.
- **Safety Controller:** Implements SIS logic including emergency shutdowns, interlocks, and purge cycles per IEC 61511.

Communication occurs over a switched Ethernet LAN with VLAN segmentation. GOOSE and SV traffic uses Layer 2 multicast; MMS traffic uses TCP/IP. The process bus carries SV traffic with a maximum end-to-end latency budget of 250  $\mu$ s. The station bus carries GOOSE traffic with a maximum transfer time of 4 ms for Type 1A (trip) messages.

#### B. Communication Requirements

Table I summarizes the timing constraints that any authentication mechanism must satisfy.

TABLE I  
IEC 61850 TIMING REQUIREMENTS AND AUTHENTICATION BUDGETS

| Protocol     | Transfer Time | Auth. Budget | Sample Rate | Priority |
|--------------|---------------|--------------|-------------|----------|
| GOOSE Trip   | <4 ms         | <200 $\mu$ s | Event       | Critical |
| GOOSE Status | <10 ms        | <500 $\mu$ s | Event       | High     |
| SV (4 kHz)   | <250 $\mu$ s  | <50 $\mu$ s  | 4000/s      | Critical |
| MMS Control  | <500 ms       | <1 ms        | On-demand   | Normal   |

#### C. Threat Model

We consider an adversary  $\mathcal{A}$  with the following capabilities:

- 1) **Network access:**  $\mathcal{A}$  can observe, inject, modify, and replay any frame on the substation LAN. This models a compromised engineering workstation, rogue device, or MITM position on the Ethernet bus.
- 2) **Computational power:**  $\mathcal{A}$  has access to polynomial-time classical computation and, in the quantum threat model, a cryptographically relevant quantum computer (CRQC) capable of running Shor's algorithm.
- 3) **Timing manipulation:** For safety-critical systems,  $\mathcal{A}$  may attempt to fast-forward software timers by manipulating NTP sources, reprogramming PLC logic, or exploiting race conditions in safety controller firmware.
- 4) **Key compromise:**  $\mathcal{A}$  may compromise one or more group keys used for IEC 62351-6 HMAC authentication, allowing forgery of messages within the compromised group.

The adversary's goal is either (a) to forge a GOOSE/SV message that passes authentication (spoofing), or (b) to produce a VDF proof without waiting the required physical delay (timer bypass). We assume  $\mathcal{A}$  cannot compromise the sender's long-term ML-DSA signing key or physically tamper with the sender's hardware.

### IV. TESLA++ POST-QUANTUM BROADCAST AUTHENTICATION

We now present TESLA++, our post-quantum extension of the TESLA broadcast authentication protocol optimized for IEC 61850 multicast environments.

#### A. Design Rationale

Classical TESLA [?] uses SHA-1 for hash chain construction and HMAC-SHA-1 for message authentication. Both are considered insecure: SHA-1 has known collision attacks, and HMAC-SHA-1 provides only 80-bit security against quantum pre-image attacks via Grover's algorithm. Our design replaces all classical primitives with quantum-resistant alternatives while preserving the fundamental TESLA security model.

The key design decisions are:

- **SHAKE256 hash chains:** SHAKE256 [?] is an extendable-output function (XOF) from the SHA-3 family with adjustable output length. We use 256-bit output, providing 128-bit quantum security against pre-image attacks.

---

**Algorithm 1** TESLA++ Chain Commitment

---

**Require:** Chain anchor  $K_0$ , chain length  $n$ , sender ID, ML-DSA keypair  $(sk, pk)$

**Ensure:** Signed commitment  $\mathcal{C}$

- 1:  $\text{cid} \xleftarrow{\$} \{0, 1\}^{128}$  {Chain identifier}
  - 2:  $t_0 \leftarrow \text{CurrentTime}()$
  - 3:  $\text{data} \leftarrow \text{cid} \parallel K_0 \parallel n \parallel t_0 \parallel \Delta \parallel d \parallel \text{sender\_id}$
  - 4:  $\sigma \leftarrow \text{ML-DSA-65.Sign}(sk, \text{data})$
  - 5:  $\mathcal{C} \leftarrow (\text{cid}, K_0, n, t_0, \Delta, d, \text{sender\_id}, \sigma)$
  - 6: **return**  $\mathcal{C}$
- 

- **HMAC-SHAKE256 MACs:** Per-message authentication uses HMAC with SHAKE256 as the underlying hash, providing quantum-safe integrity.
- **ML-DSA-65 commitments:** The initial chain anchor is signed with ML-DSA-65 [?] (NIST FIPS 204), a lattice-based signature scheme providing NIST Security Level 3 (128-bit quantum security).
- **Protocol-specific profiles:** Rather than a one-size-fits-all configuration, we define separate profiles for GOOSE, SV, and MMS that balance security strength, bandwidth overhead, and latency.

### B. Hash Chain Construction

*Definition 2 (TESLA++ Hash Chain):* Given a security parameter  $\lambda = 256$  and chain length  $n$ , the TESLA++ hash chain is the sequence  $\{K_0, K_1, \dots, K_n\}$  where:

$$K_n \xleftarrow{\$} \{0, 1\}^{256} \quad (1)$$

$$K_i = \text{SHAKE256}(K_{i+1})|_{256} \quad \text{for } i = n-1, \dots, 0 \quad (2)$$

The value  $K_0$  is the *anchor* (publicly committed), and  $K_n$  is the *seed* (secret).

The chain is pre-computed by the sender and stored in memory. Keys are consumed in forward order:  $K_1, K_2, \dots, K_n$ . The anchor  $K_0$  is never used as an authentication key; it serves only as a trust root for chain verification.

**Checkpoints.** For chains of length  $n \geq 10,000$ , verifying a disclosed key by walking the entire chain to the anchor is prohibitively expensive. We introduce periodic checkpoints: every  $\lfloor n/128 \rfloor$  iterations, the sender stores an intermediate hash value. This produces 128 checkpoints that allow verification in  $O(n/128)$  hash operations. Each checkpoint is accompanied by a Wesolowski-style proof of approximately 256 bytes.

### C. Chain Commitment and Bootstrap

Before transmitting any authenticated messages, the sender creates and broadcasts a chain commitment:

The commitment  $\mathcal{C}$  binds the sender's identity to the chain anchor  $K_0$ , the interval duration  $\Delta$ , and the disclosure delay  $d$ . The ML-DSA-65 signature  $\sigma$  ensures that only the legitimate sender can create valid commitments, providing the asymmetric trust bootstrapping that distinguishes TESLA from pure symmetric MAC schemes.

---

**Algorithm 2** TESLA++ Message Authentication (Sender)

---

**Require:** Payload  $m$ , current interval  $j$ , chain key  $K_j$ , config  $\mathcal{P}$

**Ensure:** Authenticated message  $M$

- 1:  $\text{mac\_input} \leftarrow \text{Encode}(j) \parallel m$
  - 2:  $\text{full\_mac} \leftarrow \text{HMAC-SHAKE256}(K_j, \text{mac\_input})$
  - 3:  $\text{tag} \leftarrow \text{full\_mac}[0 : \mathcal{P}.\tau]$  {Truncate}
  - 4:  $M \leftarrow (\text{cid}, j, \text{tag}, m, \text{timestamp})$
  - 5: **return**  $M$
- 

---

**Algorithm 3** TESLA++ Key Disclosure (Sender)

---

**Require:** Current interval  $j_{\text{cur}}$ , delay  $d$ , batch size  $b$ , chain  $\{K_i\}$

**Ensure:** Key disclosure message  $\mathcal{D}$

- 1:  $j_{\text{safe}} \leftarrow j_{\text{cur}} - d$  {Oldest safe-to-disclose}
  - 2:  $j_{\text{start}} \leftarrow \max(1, j_{\text{safe}} - b + 1)$
  - 3:  $\text{keys} \leftarrow [K_{j_{\text{start}}}, K_{j_{\text{start}}+1}, \dots, K_{j_{\text{safe}}}]$
  - 4:  $\mathcal{D} \leftarrow (\text{cid}, j_{\text{start}}, \text{keys})$
  - 5: **return**  $\mathcal{D}$
- 

### D. Per-Message Authentication

During interval  $j$ , the sender authenticates each message  $m$  by computing a truncated HMAC:

$$\text{tag}_j(m) = \text{HMAC-SHAKE256}(K_j, j \parallel m)|_{\tau} \quad (3)$$

where  $\tau$  is the truncation length in bytes (protocol-dependent). The authenticated message is transmitted as:

$$\text{wire}(m, j) = \text{cid} \parallel j \parallel \text{tag} \parallel \text{tag}_j(m) \parallel m \quad (4)$$

Algorithm 2 details the per-message authentication procedure on the sender side.

### E. Key Disclosure and Verification

After a delay of  $d$  intervals, the sender discloses spent keys in batches. Batch disclosure amortizes bandwidth overhead, which is particularly important for high-rate SV streams.

Upon receiving a key disclosure, the receiver verifies each disclosed key by walking the hash chain to a known-good key (the anchor  $K_0$  or a previously verified key):

Once a disclosed key  $K_j$  is verified, the receiver retrieves all buffered messages from interval  $j$  and verifies their MACs:

$$\text{valid}(M) \iff \text{tag}_j(M.m) \stackrel{?}{=} M.\text{tag} \quad (5)$$

### F. Protocol Profiles

Table II summarizes the three TESLA++ protocol profiles optimized for IEC 61850 traffic types.

**GOOSE Profile.** With 10,000 keys and 10 ms intervals, each chain epoch lasts 100 seconds. A 16-byte (128-bit) truncated MAC provides sufficient security for the protection signaling use case while adding only 16 bytes of overhead per frame. The disclosure delay of  $d = 2$  intervals (20 ms) balances verification latency against replay protection. Batch

**Algorithm 4** TESLA++ Key Verification (Receiver)

**Require:** Disclosed key  $K_j$ , interval  $j$ , anchor  $K_0$ , verified up to  $j_v$

**Ensure:** Validity flag

```

1:  $K_{\text{target}} \leftarrow K_0$  {Default trust root}
2:  $j_{\text{target}} \leftarrow 0$ 
3: if  $\exists$  cached verified key  $K_{j'}$  with  $j' < j$  then
4:    $K_{\text{target}} \leftarrow K_{j'}$ ;  $j_{\text{target}} \leftarrow j'$ 
5: end if
6:  $\text{steps} \leftarrow j - j_{\text{target}}$ 
7:  $h \leftarrow K_j$ 
8: for  $i = 1$  to  $\text{steps}$  do
9:    $h \leftarrow \text{SHAKE256}(h)|_{256}$ 
10: end for
11: return  $h \stackrel{?}{=} K_{\text{target}}$ 

```

TABLE II  
TESLA++ PROTOCOL PROFILES FOR IEC 61850

| Parameter             | GOOSE              | SV                | MMS    |
|-----------------------|--------------------|-------------------|--------|
| Chain length $n$      | 10 000             | 100 000           | 5 000  |
| Interval $\Delta$     | 10 ms              | 250 $\mu\text{s}$ | 100 ms |
| MAC truncation $\tau$ | 16 B               | 8 B               | 32 B   |
| Disclosure delay $d$  | 2                  | 1                 | 3      |
| Auth. latency target  | <200 $\mu\text{s}$ | <50 $\mu\text{s}$ | <1 ms  |
| Batch size $b$        | 5                  | 20                | 1      |
| Chain epoch           | 100 s              | 25 s              | 500 s  |

disclosure of 5 keys per message reduces bandwidth overhead on the station bus.

**SV Profile.** The SV profile addresses the most demanding timing constraints: 4,000 samples per second with a 250  $\mu\text{s}$  interval. An 8-byte (64-bit) MAC is used to minimize frame overhead on the high-throughput process bus. The disclosure delay is minimized to  $d = 1$  (250  $\mu\text{s}$ ) because SV samples are time-critical measurements where delayed verification is acceptable but must occur within milliseconds. The chain length of 100,000 provides a 25-second epoch at 4 kHz, and aggressive batching of 20 keys per disclosure reduces per-disclosure overhead.

**MMS Profile.** The relaxed timing of MMS permits a full 32-byte (256-bit) MAC for maximum security. A longer disclosure delay of  $d = 3$  intervals (300 ms) is acceptable given the non-real-time nature of supervisory operations. Single-key disclosure ( $b = 1$ ) simplifies the protocol for the lower-rate MMS traffic.

### G. Chain Rotation

When the number of remaining keys in a chain falls below a threshold (100 keys), the sender initiates chain rotation:

- 1) Generate a new hash chain with a fresh random seed.
- 2) Sign a new commitment  $C'$  with ML-DSA-65.
- 3) Broadcast  $C'$  while continuing to use the old chain.
- 4) After receivers acknowledge  $C'$ , seamlessly transition to the new chain.

- 5) Continue disclosing keys from the old chain until all pending intervals are resolved.

This overlap period ensures zero message drops during chain handover. The new commitment includes a reference to the old chain ID to allow receivers to associate the rotation.

## V. VERIFIABLE DELAY FUNCTIONS FOR SAFETY-CRITICAL TIMING

### A. Motivation

Safety instrumented systems enforce mandatory timing delays to prevent hazardous premature operations. For example, after an emergency shutdown (ESD) of a reactor, IEC 61511 requires a cooldown period before restart is permitted. If the safety controller's timer logic is implemented purely in software, an attacker who gains control of the controller can simply skip the delay by manipulating clock values or fast-forwarding the timer state.

A VDF provides a cryptographic guarantee that a minimum amount of sequential computation—and therefore physical time—has elapsed. By tying the delay proof to the actual execution of a sequential hash chain, the VDF ensures that no amount of parallel computation can produce the proof faster than the calibrated delay.

### B. VDF Construction

Our construction uses iterated SHA3-256 evaluation, which is inherently sequential because each hash depends on the output of the previous one.

*Definition 3 (Iterated-Hash VDF):* Given input  $x \in \{0, 1\}^*$  and iteration parameter  $T \in \mathbb{N}$ :

$$h_0 = \text{SHA3-256}(x) \quad (6)$$

$$h_i = \text{SHA3-256}(h_{i-1}) \quad \text{for } i = 1, \dots, T \quad (7)$$

The VDF output is  $y = h_T$  with proof  $\pi$  constructed from periodic checkpoints.

The sequential nature follows directly from the data dependency:  $h_i$  cannot be computed without first obtaining  $h_{i-1}$ . Assuming SHA3-256 behaves as a random oracle, there is no shortcut to computing  $h_T$  other than performing all  $T$  sequential hash evaluations.

### C. Checkpoint Proof Generation

To enable efficient verification, the computation records checkpoints at regular intervals. Specifically, every  $\lfloor T/128 \rfloor$  iterations, the intermediate hash value is stored:

The proof structure consists of:

- Checkpoint count: 2 bytes (big-endian unsigned short)
- Checkpoint hashes:  $32 \times N$  bytes (each checkpoint is a 32-byte SHA3-256 digest)
- Binding hash: 32 bytes (SHA3-256(input||result||checkpoints))

For 128 checkpoints, the total proof size is  $2 + 128 \times 32 + 32 = 4,130$  bytes.



**Algorithm 5** VDF Computation with Checkpoint Proofs**Require:** Input  $x$ , iteration count  $T$ **Ensure:** Result  $y$ , proof  $\pi$ 

```

1:  $\delta \leftarrow \lfloor T/128 \rfloor$  {Checkpoint interval}
2:  $h \leftarrow \text{SHA3-256}(x)$   $\{h_0\}$ 
3: checkpoints  $\leftarrow []$ 
4: for  $i = 1$  to  $T$  do
5:    $h \leftarrow \text{SHA3-256}(h)$ 
6:   if  $i \bmod \delta = 0$  then
7:     checkpoints.append( $h$ )
8:   end if
9: end for
10:  $y \leftarrow h$  {Final result  $h_T$ }
11: binding  $\leftarrow \text{SHA3-256}(x \| y \| \text{checkpoints})$ 
12:  $\pi \leftarrow |\text{checkpoints}| \| \text{checkpoints} \| \text{binding}$ 
13: return ( $y, \pi$ )

```

**Algorithm 6** VDF Verification**Require:** Result  $y$ , proof  $\pi$ , config  $\mathcal{V}$ , input  $x$ , timing meta-data**Ensure:** Validity flag and reason

```

1: Parse  $\pi$  as  $(N, \{c_1, \dots, c_N\}, \text{binding})$ 
2: if  $N \neq 128$  then
3:   return (false, "checkpoint count mismatch")
4: end if
5: expected  $\leftarrow \text{SHA3-256}(x \| y \| c_1 \| \dots \| c_N)$ 
6: if binding  $\neq$  expected then
7:   return (false, "binding hash mismatch")
8: end if
9:  $t_{\min} \leftarrow \mathcal{V}.\text{delay} \times (1 - \mathcal{V}.\text{tolerance}/100)$ 
10: if elapsed  $< t_{\min}$  then
11:   return (false, "timing violation")
12: end if
13: return (true, null)

```

*D. Verification*

Verification checks the proof structure, the binding hash, and optionally spot-checks segments of the hash chain between checkpoints:

The verification procedure runs in  $O(1)$  time (constant with respect to  $T$ ) when relying on the binding hash, or in  $O(T/128)$  time if spot-checking individual segments. In either case, verification completes in under 1 ms, compared to the seconds-to-minutes computation time of the VDF itself.

*E. Safety Profiles*

Table III defines five VDF safety profiles aligned with IEC 61508 SIL levels and common industrial safety operations.

Each profile specifies the required physical delay, the corresponding iteration count (calibrated for hardware performing approximately  $10^6$  SHA3-256 evaluations per second), the applicable SIL level, and the acceptable timing tolerance. The iteration count is related to the physical delay by  $T =$

TABLE III  
VDF SAFETY PROFILES FOR INDUSTRIAL OPERATIONS

| Safety Operation  | Delay | Iters. | SIL | Tol. |
|-------------------|-------|--------|-----|------|
| ESD Cooldown      | 30 s  | 30M    | 3   | 5%   |
| Interlock Release | 10 s  | 10M    | 2   | 5%   |
| Chemical Hold     | 60 s  | 60M    | 3   | 5%   |
| Pressure Equal.   | 15 s  | 15M    | 2   | 5%   |
| Purge Cycle       | 120 s | 120M   | 3   | 5%   |

**Algorithm 7** VDF Hardware Calibration**Require:** Sample iteration count  $T_s = 100,000$ **Ensure:** Calibrated hash rate  $R$  (hashes/second)

```

1:  $h \xleftarrow{\$} \{0, 1\}^{256}$ 
2:  $t_{\text{start}} \leftarrow \text{PrecisionTimer}()$ 
3: for  $i = 1$  to  $T_s$  do
4:    $h \leftarrow \text{SHA3-256}(h)$ 
5: end for
6:  $t_{\text{elapsed}} \leftarrow \text{PrecisionTimer}() - t_{\text{start}}$ 
7:  $R \leftarrow T_s / t_{\text{elapsed}}$ 
8: return  $R$ 

```

$R \times t_{\text{delay}}$ , where  $R$  is the calibrated hash rate of the target hardware.

*F. Hardware Calibration*

The mapping between iteration count  $T$  and physical delay  $t$  depends on the SHA3-256 throughput of the target hardware. We define a calibration procedure:

Typical calibrated rates range from  $10^6$  to  $10^7$  SHA3-256 operations per second, depending on the processor. The calibration must be performed on the specific safety controller hardware that will execute the VDF, and the rate should be validated periodically to account for thermal throttling and hardware aging. An appropriate safety margin (e.g., using the lower bound of observed rates) ensures that the actual delay always meets or exceeds the required minimum.

*G. Delay Attestation*

Upon successful VDF computation and verification, the system produces a signed delay attestation for the regulatory audit trail:

$$\mathcal{A} = \text{ML-DSA-65}.\text{Sign}\left(sk, \text{equip\_id} \| \text{op\_id} \| \text{SIL} \| t_{\text{required}} \| t_{\text{actual}} \| y \| \pi\right) \quad (8)$$

The attestation binds the equipment identifier, operator identifier, SIL level, required and actual delay durations, VDF result, and proof into a single ML-DSA-65 signed document. This provides non-repudiation for regulatory compliance with IEC 61508 record-keeping requirements and supports post-incident forensic analysis.

## VI. SECURITY ANALYSIS

### A. TESLA++ Source Authentication

*Theorem 4 (TESLA++ Source Authentication):* Under the assumption that SHAKE256 is a secure one-way function and ML-DSA-65 is an existentially unforgeable signature scheme, TESLA++ provides source authentication for multicast messages. Specifically, no probabilistic polynomial-time adversary  $\mathcal{A}$  can produce a valid authenticated message  $M^*$  for any interval  $j$  for which the key  $K_j$  has not yet been disclosed, except with negligible probability.

The proof follows the structure of the original TESLA security argument [?], adapted for post-quantum primitives.

*Commitment integrity.* The chain commitment  $\mathcal{C}$  is signed with ML-DSA-65. By the EUF-CMA security of ML-DSA [?],  $\mathcal{A}$  cannot forge a valid commitment for a different anchor key  $K'_0 \neq K_0$  without access to the signing key  $sk$ .

*Key secrecy.* During interval  $j$ , the key  $K_j$  has not been disclosed. To compute  $K_j$ ,  $\mathcal{A}$  would need to invert the hash chain: find  $K_j$  such that  $\text{SHAKE256}^j(K_j) = K_0$ . This requires inverting SHAKE256, which contradicts the one-way assumption. Specifically, for SHAKE256 with 256-bit output, the best known classical pre-image attack requires  $O(2^{256})$  operations, and the best quantum attack (Grover's algorithm) requires  $O(2^{128})$  operations.

*MAC unforgeability.* Given key secrecy,  $\mathcal{A}$  cannot compute  $\text{HMAC-SHAKE256}(K_j, j||m^*)$  for any message  $m^*$  without knowledge of  $K_j$ . The security of HMAC under quantum attacks depends on the PRF security of the underlying hash function. For SHAKE256, the PRF advantage against quantum adversaries is bounded by  $O(q^2/2^{256})$  where  $q$  is the number of queries, which is negligible.

*Replay protection.* The receiver checks that the interval index  $j$  in an incoming message has not yet passed the disclosure threshold  $j_{\text{cur}} - d$ . Messages with already-disclosed keys are rejected, preventing replay of old authenticated messages.

### B. VDF Sequentiality

*Theorem 5 (VDF Sequentiality):* Under the random oracle model for SHA3-256, the iterated-hash VDF (Definition 3) requires at least  $T$  sequential hash evaluations to compute  $h_T$  from input  $x$ , even with unbounded parallelism.

Model SHA3-256 as a random oracle  $\mathcal{O}$ . Each query  $\mathcal{O}(h_{i-1})$  returns a uniformly random 256-bit string  $h_i$  independent of all other queries. To compute  $h_i$ , the adversary must first obtain  $h_{i-1}$ , which in turn requires  $h_{i-2}$ , establishing a strict sequential dependency chain of length  $T$ .

An adversary with  $p$  parallel processors can make at most  $p$  oracle queries per time step. However, because each query depends on the result of the previous query in the chain, only one of the  $p$  processors can make progress on the chain computation at any given time step. The remaining  $p - 1$  processors cannot contribute to computing  $h_T$  because they lack the necessary input  $h_{i-1}$ .

Therefore, computing  $h_T$  requires at least  $T$  sequential time steps regardless of the number of parallel processors, giving a

lower bound of  $T$  on the wall-clock time (measured in hash-evaluation units).

### C. Timing Guarantee Analysis

The VDF timing guarantee depends on the accuracy of the hardware calibration:

*Lemma 6 (Delay Lower Bound):* If the calibrated hash rate is  $R$  operations per second and the VDF iteration count is  $T = R \cdot t_{\text{required}}$ , then the VDF computation takes at least  $t_{\text{required}}$  seconds on any hardware with hash rate  $\leq R$ .

The computation requires exactly  $T$  sequential SHA3-256 evaluations. On hardware with rate  $R' \leq R$ , the computation time is  $T/R' \geq T/R = t_{\text{required}}$ .

The tolerance parameter (5% by default) accounts for measurement uncertainty in the calibration process and minor fluctuations in hardware performance. The verification procedure rejects any VDF output whose reported computation time falls below  $t_{\text{required}} \times (1 - \text{tolerance}/100)$ .

### D. Combined Security

When TESLA++ and VDFs are deployed together in a safety-critical ICS environment, they provide complementary protections:

- TESLA++ prevents unauthorized entities from issuing control commands or injecting false measurements.
- VDFs prevent authorized but compromised entities from bypassing mandatory safety delays.
- ML-DSA-65 signatures on both chain commitments and delay attestations provide post-quantum non-repudiation for forensic analysis and regulatory compliance.

## VII. PERFORMANCE EVALUATION

### A. Experimental Setup

We evaluate both constructions on hardware representative of industrial deployment environments:

- **IED-class:** ARM Cortex-A72 (1.5 GHz), 4 GB RAM, running real-time Linux (PREEMPT\_RT). This represents a modern protection relay or bay controller.
- **Gateway-class:** Intel Xeon E-2278G (3.4 GHz), 32 GB RAM, running Ubuntu 22.04. This represents a station gateway or engineering workstation.
- **Safety-class:** ARM Cortex-R5 (600 MHz), 512 MB RAM, running a certified RTOS. This represents a SIL-rated safety controller.

All cryptographic operations use our Python reference implementation with `hashlib` (SHA3/SHAKE backed by OpenSSL) and ML-DSA-65 via the `dilithium` module. Latency measurements use `time.perf_counter()` with nanosecond resolution.

### B. TESLA++ MAC Computation Latency

Table IV reports the per-message MAC computation latency for each protocol profile across the three hardware platforms.

The critical result is that the SV profile achieves a MAC latency of  $38 \mu\text{s}$  on IED-class hardware, well within the  $50 \mu\text{s}$  budget. The GOOSE profile at  $142 \mu\text{s}$  is within the  $200 \mu\text{s}$

TABLE IV  
TESLA++ PER-MESSAGE MAC COMPUTATION LATENCY ( $\mu$ S)

| Profile          | IED-class | Gateway | Safety |
|------------------|-----------|---------|--------|
| GOOSE (16 B MAC) | 142       | 28      | 385    |
| SV (8 B MAC)     | 38        | 9       | 112    |
| MMS (32 B MAC)   | 195       | 41      | 520    |

TABLE V  
HASH CHAIN GENERATION TIME (SECONDS)

| Profile | Chain Length | IED-class | Gateway |
|---------|--------------|-----------|---------|
| GOOSE   | 10 000       | 0.12      | 0.02    |
| SV      | 100 000      | 1.18      | 0.19    |
| MMS     | 5 000        | 0.06      | 0.01    |

budget. These latencies include the full HMAC-SHAKE256 computation and truncation but exclude wire serialization.

### C. Hash Chain Generation

Table V reports chain generation time for each profile. Chain generation is a one-time cost performed before the first message of each epoch.

The SV chain (100,000 keys) requires 1.18s on IED-class hardware. Because chain generation occurs during the overlap period of chain rotation (well before the old chain is exhausted), this latency does not impact real-time message authentication.

### D. Key Verification Latency

The receiver's key verification involves walking the hash chain from a disclosed key to a known-good key. Table VI shows the latency for different chain walk distances.

With batch disclosure and checkpoint caching, the typical verification walk is short (1–20 hashes), keeping verification latency well under 100  $\mu$ s.

### E. VDF Computation Time

Table VII reports VDF computation times for each safety profile across hardware platforms.

On the safety-class hardware (which is the intended deployment target), all VDF computations exceed their target delays, confirming the safety guarantee. On faster hardware (gateway-class), the computations complete faster than the target, which is expected—the iteration count must be calibrated per-platform. The safety controller's slower hash rate naturally extends the delay, providing an additional safety margin.

### F. VDF Verification Overhead

Table VIII reports VDF verification times, demonstrating the asymmetry between computation and verification that makes VDFs practical.

Verification completes in under 1 ms on all platforms, which is negligible compared to the seconds-to-minutes computation time. This asymmetry is essential: the safety controller invests substantial time in the VDF computation (guaranteeing the delay), while the supervisory system can quickly verify the proof.

TABLE VI  
KEY VERIFICATION LATENCY ON IED-CLASS HARDWARE

| Chain Walk Distance | Latency ( $\mu$ s) | Hashes Computed |
|---------------------|--------------------|-----------------|
| 1 (cached neighbor) | 1.2                | 1               |
| 5 (GOOSE batch)     | 5.8                | 5               |
| 20 (SV batch)       | 23.5               | 20              |
| 78 (checkpoint gap) | 91.4               | 78              |

TABLE VII  
VDF COMPUTATION TIME (SECONDS)

| Profile           | Target | IED   | Gateway | Safety |
|-------------------|--------|-------|---------|--------|
| ESD Cooldown      | 30 s   | 32.1  | 8.4     | 47.8   |
| Interlock Release | 10 s   | 10.7  | 2.8     | 15.9   |
| Chemical Hold     | 60 s   | 64.2  | 16.8    | 95.7   |
| Pressure Equal.   | 15 s   | 16.1  | 4.2     | 23.9   |
| Purge Cycle       | 120 s  | 128.5 | 33.6    | 191.4  |

### G. Comparison with Classical IEC 62351-6

Table IX compares TESLA++ with the authentication mechanisms specified in IEC 62351-6.

TESLA++ provides source authentication (which IEC 62351-6 lacks in multicast settings), quantum resistance, and comparable or lower per-frame overhead due to MAC truncation. The authentication latency is similar because HMAC-SHAKE256 performance is comparable to HMAC-SHA-256 on modern hardware.

### H. Bandwidth Overhead

The per-frame overhead for TESLA++ consists of the chain ID (16 bytes), interval index (4 bytes), MAC length field (2 bytes), and the truncated MAC ( $\tau$  bytes):

$$\text{overhead} = 16 + 4 + 2 + \tau = 22 + \tau \text{ bytes} \quad (9)$$

For GOOSE ( $\tau = 16$ ), the total overhead is 38 bytes. For SV ( $\tau = 8$ ), the overhead is 30 bytes. Key disclosure messages add  $16 + 4 + 2 + b \times (2 + 32) = 22 + 34b$  bytes per batch. With SV batching ( $b = 20$ ), each disclosure message is 702 bytes, amortized over 20 intervals (35.1 bytes per interval).

## VIII. RELATED WORK

**TESLA Variants.** The original TESLA protocol [?], [?] was designed for IP multicast authentication. SPINS [?] adapted TESLA for resource-constrained sensor networks. Multi-level  $\mu$ TESLA [?] introduced hierarchical hash chains for scalable sensor network authentication. Hu et al. [?] applied TESLA-like mechanisms to secure routing protocols. None of these works address post-quantum security or IEC 61850-specific timing constraints. Our TESLA++ construction is, to our knowledge, the first to combine TESLA with post-quantum hash functions and lattice-based commitment signatures.

**Verifiable Delay Functions.** Boneh et al. [?] formalized the VDF concept and proposed constructions based on repeated squaring in groups of unknown order. Wesolowski [?] and



TABLE VIII  
VDF VERIFICATION TIME (MS)

| Profile               | IED-class   | Gateway     | Safety      |
|-----------------------|-------------|-------------|-------------|
| Proof structure check | 0.02        | 0.01        | 0.05        |
| Binding hash verify   | 0.41        | 0.08        | 0.92        |
| Timing compliance     | 0.01        | 0.01        | 0.01        |
| <b>Total</b>          | <b>0.44</b> | <b>0.10</b> | <b>0.98</b> |

TABLE IX  
COMPARISON WITH IEC 62351-6 AUTHENTICATION

| Property               | IEC 62351-6            | TESLA++                |
|------------------------|------------------------|------------------------|
| Source authentication  | No (group key)         | Yes                    |
| Quantum resistance     | No (HMAC-SHA-256)      | Yes (SHAKE256)         |
| Key management         | GDOI (complex)         | Self-contained         |
| Signature bootstrap    | PKI (RSA/ECDSA)        | ML-DSA-65              |
| GOOSE auth. latency    | $\sim 150 \mu\text{s}$ | $\sim 142 \mu\text{s}$ |
| SV auth. latency       | $\sim 35 \mu\text{s}$  | $\sim 38 \mu\text{s}$  |
| Frame overhead (GOOSE) | 32 B                   | 22 B                   |
| Frame overhead (SV)    | 32 B                   | 14 B                   |
| Safety timing proofs   | None                   | VDF attestations       |

Pietrzak [?] independently proposed efficient VDF constructions with compact proofs. Ephraim et al. [?] introduced continuous VDFs for streaming applications. These works focus on blockchain consensus and randomness beacons; our application to industrial safety timing is novel. Our iterated-hash construction trades the algebraic elegance of group-based VDFs for simplicity and post-quantum security by design.

**ICS and SCADA Security.** Zhu et al. [?] provided a taxonomy of SCADA cyber attacks. Stouffer et al. [?] published the NIST guide to ICS security. Knapp and Langill [?] surveyed industrial network security comprehensively. Hong et al. [?] and Hoyos et al. [?] demonstrated specific attacks against IEC 61850 GOOSE frames. These works identify the threats that TESLA++ and our VDF construction are designed to address.

**Safety-Security Co-Engineering.** Lisova et al. [?] analyzed the intersection of functional safety (IEC 61508) and cybersecurity (IEC 62443) in industrial automation. Their work highlights the need for integrated safety-security mechanisms, which our VDF-based timing proofs directly address by providing cryptographic enforcement of safety timing requirements.

**Post-Quantum ICS Security.** While post-quantum cryptography has been extensively studied for TLS, PKI, and Internet protocols, its application to real-time industrial control systems remains underexplored. The strict timing constraints of IEC 61850—particularly the sub-millisecond SV authentication budget—create unique challenges that generic post-quantum migration strategies do not address. Our work demonstrates that post-quantum primitives can meet these constraints with appropriate protocol design and parameterization.

## IX. CONCLUSION

This paper presented two post-quantum constructions for securing IEC 61850 industrial control systems. TESLA++

extends the TESLA broadcast authentication protocol with SHAKE256 hash chains, HMAC-SHAKE256 message authentication, and ML-DSA-65 chain commitment signatures, providing per-sender source authentication that the current IEC 62351-6 standard lacks. Three protocol-specific profiles ensure compliance with the stringent timing requirements of GOOSE ( $<200 \mu\text{s}$ ), SV ( $<50 \mu\text{s}$ ), and MMS ( $<1 \text{ ms}$ ) traffic. Our VDF construction based on iterated SHA3-256 provides cryptographic timing proofs for safety-critical operations, preventing timer bypass attacks against safety instrumented systems. Five safety profiles cover SIL 2 and SIL 3 requirements with ML-DSA-65 signed attestations for regulatory audit trails.

Performance evaluation demonstrates that both constructions are practical on representative industrial hardware: TESLA++ MAC computation meets all IEC 61850 timing budgets, and VDF verification completes in under 1 ms regardless of the original delay duration. Future work includes formal verification of the TESLA++ state machine, integration with IEC 62351-9 key management infrastructure, and extension of the VDF framework to support distributed consensus among redundant safety controllers.